

Chapter 50 — Math / Number Theory

0. What You Will Learn in This Chapter

Math and Number Theory are important for interview problems that involve:

- divisibility
- primes
- factors
- greatest common divisor
- least common multiple
- modular arithmetic
- fast exponentiation
- combinatorics foundations
- integer overflow
- digit math
- parity
- number transformations
- randomization/probability previews
- game theory foundations

DE Shaw and similar algorithm-heavy interviews can include mathy reasoning, so this chapter is important.

By the end of this chapter, you should understand:

- GCD and LCM
- Euclidean algorithm
- Prime checking
- Sieve of Eratosthenes
- Prime factorization
- Modular arithmetic
- Fast exponentiation
- Modular exponentiation
- Modular inverse preview
- Divisibility tricks
- Overflow-safe multiplication
- Integer square root logic
- Counting factors
- Math transformations
- Common LeetCode math patterns

Core problems covered in this chapter:

1. Greatest Common Divisor of Strings
2. Find GCD of Array
3. LCM and GCD basics
4. Count Primes
5. Ugly Number
6. Ugly Number II
7. Pow(x, n)
8. Super Pow, preview
9. Excel Sheet Column Number
10. Excel Sheet Column Title
11. Happy Number

- 12. Add Digits
 - 13. Sqrt(x)
 - 14. Valid Perfect Square
 - 15. Factorial Trailing Zeroes
 - 16. Integer Break, math view
 - 17. Water and Jug Problem
 - 18. Mirror Reflection
 - 19. Rectangle Overlap / Geometry basics
 - 20. Convex Polygon / geometry preview
 - 21. Modular inverse preview
-

1. Why Math Matters in DSA Interviews

Math problems often look small but require insight.

Examples:

Find GCD of two numbers.
Count primes less than n .
Compute x^n efficiently.
Find trailing zeroes in factorial.
Check whether water jug target is possible.

These problems test whether you can convert a statement into a mathematical condition.

For DE Shaw-level prep, you should be comfortable with math-based reasoning, not just code patterns.

2. Core Number Theory Topics

Topic	Used For
GCD	divisibility, reducing fractions, grouping
LCM	cycle alignment, repeated events
Primes	factorization, counting, divisibility
Sieve	counting primes efficiently
Modular arithmetic	large counts, exponentiation
Fast power	powers in $O(\log n)$
Factorization	divisor problems
Digital root	repeated digit sum
Integer sqrt	binary search/math

Topic	Used For
Parity	odd/even reasoning
Pigeonhole principle	proof-style problems

3. GCD — Greatest Common Divisor

The GCD of two numbers is the largest number that divides both.

Example:

$\text{gcd}(12, 18) = 6$

Divisors of 12:

1, 2, 3, 4, 6, 12

Divisors of 18:

1, 2, 3, 6, 9, 18

Common largest:

6

4. Euclidean Algorithm

Key identity:

$\text{gcd}(a, b) = \text{gcd}(b, a \% b)$

until:

$b = 0$

Then answer is:

a

Code

```
int gcd(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }

    return a;
}
```

Or use built-in:

`std::gcd(a, b)`

from C++17.

5. LCM — Least Common Multiple

LCM is smallest positive number divisible by both a and b.

Formula:

```
lcm(a, b) = a / gcd(a, b) * b
```

Use division first to avoid overflow.

```
long long lcm(long long a, long long b) {  
    return a / gcd(a, b) * b;  
}
```

6. GCD and LCM Relationship

Important identity:

```
gcd(a, b) * lcm(a, b) = a * b
```

Use:

```
lcm = a / gcd * b
```

not:

```
a * b / gcd
```

because $a * b$ may overflow.

7. Full Problem Walkthrough 1 — LeetCode 1071: Greatest Common Divisor of Strings

7.1 Problem Summary

For two strings str1 and str2, find largest string x such that x divides both strings.

A string x divides s if s is made by repeating x.

Example:

```
str1 = "ABCABC"  
str2 = "ABC"
```

Answer:

```
"ABC"
```

7.2 Key Insight

If two strings have a common repeating base, then:

```
str1 + str2 == str2 + str1
```

If not, no common divisor string exists.

Length of answer is:

```
gcd(str1.size(), str2.size())
```

7.3 C++ Code

```
class Solution {
public:
    string gcdOfStrings(string str1, string str2) {
        if (str1 + str2 != str2 + str1) {
            return "";
        }

        int len = gcd((int)str1.size(), (int)str2.size());

        return str1.substr(0, len);
    }
};
```

7.4 Complexity

Time:

$O(n + m)$

Space:

$O(n + m)$

because of concatenation.

8. Full Problem Walkthrough 2 — Find GCD of Array

8.1 Problem Summary

Given array `nums`, return GCD of smallest and largest number.

8.2 C++ Code

```
class Solution {
public:
    int findGCD(vector<int>& nums) {
        int mn = *min_element(nums.begin(), nums.end());
        int mx = *max_element(nums.begin(), nums.end());

        return gcd(mn, mx);
    }
};
```

8.3 General Array GCD

```
int arrayGCD(vector<int>& nums) {
    int g = 0;

    for (int x : nums) {
```

```
        g = gcd(g, x);
    }

    return g;
}
```

9. Prime Numbers

A prime number has exactly two positive divisors:

1 and itself

Examples:

2, 3, 5, 7, 11, 13

Non-primes:

1 is not prime
0 is not prime
negative numbers are not prime
4, 6, 8, 9 are composite

10. Prime Checking

To check if n is prime, test divisors up to $\sqrt{n}$.

Why?

If:

$n = a * b$

then at least one of a or b is $\leq \sqrt{n}$.

[Code](#)

```
bool isPrime(int n) {
    if (n < 2) {
        return false;
    }

    for (long long d = 2; d * d <= n; d++) {
        if (n % d == 0) {
            return false;
        }
    }

    return true;
}
```

Optimization:

- handle 2 separately
- then check odd divisors only

```

bool isPrime(int n) {
    if (n < 2) {
        return false;
    }

    if (n == 2) {
        return true;
    }

    if (n % 2 == 0) {
        return false;
    }

    for (long long d = 3; d * d <= n; d += 2) {
        if (n % d == 0) {
            return false;
        }
    }

    return true;
}

```

11. Sieve of Eratosthenes

Use sieve to find all primes up to n .

Idea

Start with all numbers marked prime.

For each prime p , mark multiples of p as non-prime.

Start marking from:

$p * p$

because smaller multiples were already marked.

Code

```

vector<bool> sieve(int n) {
    vector<bool> isPrime(n + 1, true);

    if (n >= 0) {
        isPrime[0] = false;
    }

    if (n >= 1) {
        isPrime[1] = false;
    }

    for (long long p = 2; p * p <= n; p++) {
        if (isPrime[p]) {
            for (long long multiple = p * p; multiple <= n; multiple

```

```

+= p) {
    isPrime[multiple] = false;
}
}
return isPrime;
}

```

Complexity:

$O(n \log \log n)$

12. Full Problem Walkthrough 3 — LeetCode 204: Count Primes

12.1 Problem Summary

Count primes less than n .

Example:

$n = 10$

Primes less than 10:

2, 3, 5, 7

Answer:

4

12.2 C++ Code

```

class Solution {
public:
    int countPrimes(int n) {
        if (n <= 2) {
            return 0;
        }

        vector<bool> isPrime(n, true);

        isPrime[0] = false;
        isPrime[1] = false;

        for (long long p = 2; p * p < n; p++) {
            if (isPrime[p]) {
                for (long long multiple = p * p; multiple < n;
multiple += p) {
                    isPrime[multiple] = false;
                }
            }
        }
    }
}

```

```

        int count = 0;

        for (bool prime : isPrime) {
            if (prime) {
                count++;
            }
        }

        return count;
    }
};

```

13. Prime Factorization

Prime factorization breaks n into prime powers.

Example:

$84 = 2 * 2 * 3 * 7$

Code:

```

vector<pair<int, int>> factorize(int n) {
    vector<pair<int, int>> factors;

    for (int p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            int count = 0;

            while (n % p == 0) {
                n /= p;
                count++;
            }

            factors.push_back({p, count});
        }
    }

    if (n > 1) {
        factors.push_back({n, 1});
    }

    return factors;
}

```

Complexity:

$O(\sqrt{n})$

14. Counting Divisors From Factorization

If:

$$n = p_1^a * p_2^b * p_3^c$$

Number of divisors:

$$(a + 1)(b + 1)(c + 1)$$

Example:

$$12 = 2^2 * 3^1$$

Number of divisors:

$$(2 + 1)(1 + 1) = 6$$

Divisors:

1, 2, 3, 4, 6, 12

15. Modular Arithmetic

Many problems ask answer modulo:

1,000,000,007

because answer can be huge.

Rules:

Addition

$$(a + b) \% \text{MOD}$$

Subtraction

$$(a - b + \text{MOD}) \% \text{MOD}$$

Multiplication

$$(a * b) \% \text{MOD}$$

Use long long:

$$(1LL * a * b) \% \text{MOD}$$

Division

Modulo division is not normal division.

Need modular inverse when MOD is prime.

16. Fast Exponentiation

Compute:

$$x^n$$

in $O(\log n)$.

Idea:

```
x^10 = x^5 * x^5  
x^5 = x * x^4
```

Use binary exponentiation.

Code

```
long long fastPower(long long x, long long n) {  
    long long result = 1;  
  
    while (n > 0) {  
        if (n & 1) {  
            result *= x;  
        }  
  
        x *= x;  
        n >>= 1;  
    }  
  
    return result;  
}
```

17. Modular Fast Exponentiation

```
long long modPow(long long base, long long exp, long long MOD) {  
    long long result = 1 % MOD;  
  
    base %= MOD;  
  
    while (exp > 0) {  
        if (exp & 1) {  
            result = result * base % MOD;  
        }  
  
        base = base * base % MOD;  
        exp >>= 1;  
    }  
  
    return result;  
}
```

18. Full Problem Walkthrough 4 — LeetCode 50: Pow(x, n)

18.1 Problem Summary

Implement:

`pow(x, n)`

n may be negative.

18.2 Key Details

If n is negative:

$x^n = 1 / x^{-n}$

But beware:

`n = INT_MIN`

-n overflows if n is int.

Use long long.

18.3 C++ Code

```
class Solution {
public:
    double myPow(double x, int n) {
        long long exp = n;

        if (exp < 0) {
            x = 1.0 / x;
            exp = -exp;
        }

        double result = 1.0;

        while (exp > 0) {
            if (exp & 1) {
                result *= x;
            }

            x *= x;
            exp >>= 1;
        }

        return result;
    }
};
```

18.4 Complexity

Time:

$O(\log n)$

Space:

$O(1)$

19. Modular Inverse Preview

If MOD is prime, modular inverse of a is:

```
a^(MOD - 2) mod MOD
```

using Fermat's little theorem.

```
long long modInverse(long long a, long long MOD) {  
    return modPow(a, MOD - 2, MOD);  
}
```

This is used for:

- combinations modulo prime
- modular division
- probability modulo problems

More will appear in combinatorics chapter.

20. Full Problem Walkthrough 5 — Ugly Number

20.1 Problem Summary

An ugly number has only prime factors:

```
2, 3, 5
```

Return true if n is ugly.

20.2 C++ Code

```
class Solution {  
public:  
    bool isUgly(int n) {  
        if (n <= 0) {  
            return false;  
        }  
  
        vector<int> primes = {2, 3, 5};  
  
        for (int p : primes) {  
            while (n % p == 0) {  
                n /= p;  
            }  
        }  
  
        return n == 1;  
    }  
};
```

21. Full Problem Walkthrough 6 — LeetCode 264: Ugly Number II

21.1 Problem Summary

Return nth ugly number.

Ugly numbers are positive numbers whose prime factors are only:

2, 3, 5

21.2 DP / Three Pointer Idea

Every ugly number comes from:

```
previous ugly * 2
previous ugly * 3
previous ugly * 5
```

Maintain pointers:

i2, i3, i5

21.3 C++ Code

```
class Solution {
public:
    int nthUglyNumber(int n) {
        vector<int> ugly(n);

        ugly[0] = 1;

        int i2 = 0;
        int i3 = 0;
        int i5 = 0;

        for (int i = 1; i < n; i++) {
            int next2 = ugly[i2] * 2;
            int next3 = ugly[i3] * 3;
            int next5 = ugly[i5] * 5;

            int next = min({next2, next3, next5});

            ugly[i] = next;

            if (next == next2) {
                i2++;
            }

            if (next == next3) {
                i3++;
            }

            if (next == next5) {
                i5++;
            }
        }

        return ugly[n - 1];
    }
};
```

```
    }  
};
```

21.4 Why Increment All Matching Pointers?

Example:

```
6 = 2 * 3 = 3 * 2
```

To avoid duplicates, increment both pointers that produce 6.

22. Full Problem Walkthrough 7 — LeetCode 202: Happy Number

22.1 Problem Summary

Replace number by sum of squares of digits.

If eventually reaches 1, happy.

Otherwise cycle.

22.2 Pattern

Cycle detection.

Use hash set or fast/slow.

22.3 C++ Code

```
class Solution {  
public:  
    int nextNumber(int n) {  
        int sum = 0;  
  
        while (n > 0) {  
            int d = n % 10;  
            sum += d * d;  
            n /= 10;  
        }  
  
        return sum;  
    }  
  
    bool isHappy(int n) {  
        unordered_set<int> seen;  
  
        while (n != 1 && !seen.count(n)) {  
            seen.insert(n);  
            n = nextNumber(n);  
        }  
  
        return n == 1;  
    }  
};
```

23. Full Problem Walkthrough 8 — LeetCode 258: Add Digits

23.1 Problem Summary

Repeatedly add digits until one digit remains.

Example:

```
38 -> 3 + 8 = 11 -> 1 + 1 = 2
```

23.2 Digital Root Formula

For positive n:

```
answer = 1 + (n - 1) % 9
```

For n = 0:

```
0
```

23.3 C++ Code

```
class Solution {
public:
    int addDigits(int num) {
        if (num == 0) {
            return 0;
        }

        return 1 + (num - 1) % 9;
    }
};
```

24. Full Problem Walkthrough 9 — LeetCode 69: Sqrt(x)

24.1 Problem Summary

Return floor square root of x.

24.2 Binary Search

Need largest m such that:

```
m * m <= x
```

Avoid overflow by using:

```
m <= x / m
```

24.3 C++ Code

```
class Solution {
public:
    int mySqrt(int x) {
        int left = 0;
```

```

int right = x;
int answer = 0;

while (left <= right) {
    int mid = left + (right - left) / 2;

    if (mid == 0 || mid <= x / mid) {
        answer = mid;
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return answer;
}
};

```

25. Full Problem Walkthrough 10 — LeetCode 367: Valid Perfect Square

25.1 Problem Summary

Return true if num is perfect square.

25.2 Binary Search

```

class Solution {
public:
    bool isPerfectSquare(int num) {
        long long left = 1;
        long long right = num;

        while (left <= right) {
            long long mid = left + (right - left) / 2;
            long long square = mid * mid;

            if (square == num) {
                return true;
            }

            if (square < num) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return false;
    }
};

```

```
}  
};
```

26. Full Problem Walkthrough 11 — LeetCode 172: Factorial Trailing Zeroes

26.1 Problem Summary

Return number of trailing zeroes in $n!$.

26.2 Key Insight

Trailing zero comes from factor:

$$10 = 2 * 5$$

There are always more 2s than 5s.

So count factors of 5.

26.3 Formula

$$n / 5 + n / 25 + n / 125 + \dots$$

26.4 C++ Code

```
class Solution {  
public:  
    int trailingZeroes(int n) {  
        int count = 0;  
  
        while (n > 0) {  
            n /= 5;  
            count += n;  
        }  
  
        return count;  
    }  
};
```

27. Full Problem Walkthrough 12 — LeetCode 343: Integer Break

27.1 Problem Summary

Break integer n into at least two positive integers to maximize product.

27.2 Math Insight

Use as many 3s as possible.

Why?

3 gives best product growth.

Rules:

- If $n == 2$, answer 1.
- If $n == 3$, answer 2.
- While $n > 4$, multiply answer by 3 and reduce n by 3.
- Multiply remaining n .

27.3 C++ Code

```
class Solution {
public:
    int integerBreak(int n) {
        if (n == 2) {
            return 1;
        }

        if (n == 3) {
            return 2;
        }

        int product = 1;

        while (n > 4) {
            product *= 3;
            n -= 3;
        }

        return product * n;
    }
};
```

28. Full Problem Walkthrough 13 — LeetCode 365: Water and Jug Problem

28.1 Problem Summary

Given jug capacities x and y , determine if target liters can be measured.

28.2 Bézout's Identity

You can measure target if:

$\text{target} \leq x + y$

and:

target is divisible by $\text{gcd}(x, y)$

28.3 C++ Code

```
class Solution {
public:
```

```

bool canMeasureWater(int x, int y, int target) {
    if (target > x + y) {
        return false;
    }

    if (target == 0) {
        return true;
    }

    return target % gcd(x, y) == 0;
}
};

```

29. Full Problem Walkthrough 14 — LeetCode 858: Mirror Reflection

29.1 Problem Summary

A laser ray reflects in square room.

Given p and q , return receptor number.

29.2 Math Insight

Find least common multiple of p and q .

Number of room extensions:

$$m = \text{lcm}(p, q) / p$$

Number of vertical hits:

$$n = \text{lcm}(p, q) / q$$

Then determine receptor by parity.

Simpler approach:

Reduce p and q by powers of 2 until at least one is odd.

29.3 C++ Code

```

class Solution {
public:
    int mirrorReflection(int p, int q) {
        while (p % 2 == 0 && q % 2 == 0) {
            p /= 2;
            q /= 2;
        }

        if (p % 2 == 1 && q % 2 == 1) {
            return 1;
        }

        if (p % 2 == 1 && q % 2 == 0) {
            return 0;
        }
    }
};

```

```
        return 2;
    }
};
```

30. Geometry Basics Preview

Some math problems involve coordinates.

Common geometry checks:

Rectangle Overlap

Two rectangles overlap if:

```
max(left1, left2) < min(right1, right2)
```

and:

```
max(bottom1, bottom2) < min(top1, top2)
```

Strict inequality means positive area overlap.

Distance Squared

Avoid floating point:

```
dx * dx + dy * dy
```

Collinearity

Three points are collinear if cross product is 0:

```
(x2 - x1)(y3 - y1) - (y2 - y1)(x3 - x1) = 0
```

More geometry can be added later if needed.

31. Common Math Edge Cases

31.1 Zero

GCD with zero:

```
gcd(a, 0) = a
```

Power:

```
x^0 = 1
```

31.2 Negative Numbers

Modulo with negatives can be tricky.

Use:

```
(x % MOD + MOD) % MOD
```

31.3 Overflow

Use long long for:

- multiplication
- square checks
- lcm
- power

31.4 INT_MIN

Negating INT_MIN overflows as int.

Use long long.

31.5 $n < 2$ for Prime

0 and 1 are not prime.

32. Common Math Mistakes

Mistake 1 — Checking Divisors Up to n

Prime check only needs up to $\sqrt{n}$.

Mistake 2 — Overflow in $\text{mid} * \text{mid}$

Use:

```
mid <= x / mid
```

or long long.

Mistake 3 — LCM Overflow

Use:

```
a / gcd(a, b) * b
```

Mistake 4 — Wrong Mod Subtraction

Use:

```
(a - b + MOD) % MOD
```

Mistake 5 — Forgetting Special Cases

Examples:

```
n = 0
n = 1
negative exponent
target = 0
```

33. Interview Explanation Templates

33.1 GCD

I use the Euclidean algorithm because $\text{gcd}(a, b)$ equals $\text{gcd}(b, a \bmod b)$, and it terminates when the remainder becomes zero.

33.2 Sieve

I mark all numbers as prime initially, then for every prime p , I mark multiples from p squared onward as composite.

33.3 Fast Power

I use binary exponentiation. If the current exponent bit is set, I multiply the answer by the base. Then I square the base and shift the exponent.

33.4 Trailing Zeroes

A trailing zero comes from factor 10, which is 2 times 5. Since factorials have more 2s than 5s, I only count factors of 5.

33.5 Water Jug

By Bézout's identity, any measurable amount must be a multiple of $\gcd(x, y)$, and it cannot exceed $x + y$.

34. Complexity Patterns

Operation / Problem	Time	Space
GCD	$O(\log \min(a, b))$	$O(1)$
LCM	$O(\log \min(a, b))$	$O(1)$
Prime check	$O(\sqrt{n})$	$O(1)$
Sieve	$O(n \log \log n)$	$O(n)$
Factorization	$O(\sqrt{n})$	$O(1)$ / output
Fast power	$O(\log n)$	$O(1)$
Modular inverse	$O(\log \text{MOD})$	$O(1)$
Sqrt binary search	$O(\log n)$	$O(1)$
Trailing zeroes	$O(\log n \text{ base } 5)$	$O(1)$

35. Expanded Practice Set

35.1 Must-Do Easy / Medium

- 22. Greatest Common Divisor of Strings
- 23. Find GCD of Array
- 24. Count Primes
- 25. Ugly Number
- 26. Ugly Number II
- 27. Pow(x, n)
- 28. Happy Number
- 29. Add Digits
- 30. Sqrt(x)
- 31. Valid Perfect Square
- 32. Factorial Trailing Zeroes
- 33. Excel Sheet Column Number
- 34. Excel Sheet Column Title

35.2 Strong Interview Practice

- 35. Integer Break
- 36. Water and Jug Problem
- 37. Mirror Reflection
- 38. Super Pow
- 39. Rectangle Overlap

- 40. Check If It Is a Good Array
- 41. Minimum Moves to Equal Array Elements II
- 42. Reach a Number
- 43. Bulb Switcher

35.3 Hard / Stretch

- 44. Modular inverse problems
 - 45. Chinese Remainder Theorem preview
 - 46. Euler Totient preview
 - 47. Miller-Rabin primality preview
 - 48. Advanced geometry orientation problems
 - 49. Random point in circle / rectangle
 - 50. Fast combinatorics modulo prime
-

36. Suggested Study Order

Step 1 — GCD / LCM

Solve:

- 51. GCD of Strings
- 52. Find GCD of Array
- 53. Water Jug Problem

Goal:

Master divisibility reasoning.

Step 2 — Primes

Solve:

- 54. Count Primes
- 55. Ugly Number
- 56. Prime factorization exercises

Goal:

Understand primality and factors.

Step 3 — Modular Arithmetic

Solve:

- 57. Pow(x, n)
- 58. Super Pow
- 59. modular exponentiation exercises

Goal:

Master fast power.

Step 4 — Digit / Number Math

Solve:

- 60. Add Digits
- 61. Happy Number
- 62. Excel Column Number
- 63. Excel Column Title

Goal:

Practice base conversion and digit logic.

Step 5 — Binary Search Math

Solve:

- 64. Sqrt(x)

65. Valid Perfect Square
Goal:
Avoid overflow in math search.

37. Pattern Decision Table

Problem Clue	Math Technique
largest common divisor	GCD
repeated cycles align	LCM
count primes up to n	sieve
check prime	trial division to sqrt
factor decomposition	prime factorization
huge power	fast exponentiation
modulo division	modular inverse
trailing zeroes in factorial	count factors of 5
repeated digit sum	digital root
square root integer	binary search
water jug possible	Bézout / gcd
ugly numbers	prime factor filtering / DP
overflow risk	long long / divide before multiply

38. Mini Problem 1 — Excel Sheet Column Number

A -> 1
B -> 2
Z -> 26
AA -> 27

This is base-26 conversion.

```
class Solution {
public:
    int titleToNumber(string columnTitle) {
        int result = 0;
```

```

        for (char c : columnTitle) {
            result = result * 26 + (c - 'A' + 1);
        }

        return result;
    }
};

```

39. Mini Problem 2 — Excel Sheet Column Title

Reverse conversion.

```

class Solution {
public:
    string convertToTitle(int columnNumber) {
        string result;

        while (columnNumber > 0) {
            columnNumber--;

            int rem = columnNumber % 26;

            result.push_back('A' + rem);

            columnNumber /= 26;
        }

        reverse(result.begin(), result.end());

        return result;
    }
};

```

Why subtract 1?

Because Excel columns are 1-indexed, not 0-indexed.

40. Mini Problem 3 — Check Good Array

Problem:

Given nums, return true if some integer combination can make 1.

By Bézout:

Possible iff gcd of all nums is 1.

```

class Solution {
public:

```

```

bool isGoodArray(vector<int>& nums) {
    int g = 0;

    for (int x : nums) {
        g = gcd(g, x);
    }

    return g == 1;
}
};

```

41. Mini Problem 4 — Bulb Switcher

Problem:

n bulbs toggled in rounds.

A bulb remains on if toggled odd number of times.

Bulb i is toggled once for each divisor of i .

Only perfect squares have odd number of divisors.

Answer:

```

floor(sqrt(n))
class Solution {
public:
    int bulbSwitch(int n) {
        return sqrt(n);
    }
};

```

42. Debugging Math Problems

If a math solution fails, check:

66. Did you handle 0 and 1?
67. Did you handle negative values?
68. Is multiplication overflowing?
69. Should you use long long?
70. Is modulo subtraction negative?
71. Did you prove the formula?
72. Is the range inclusive or exclusive?
73. Does the problem ask less than n or less than/equal n ?
74. Did you count factors correctly?
75. Is integer division truncating unexpectedly?

43. What to Master Before Chapter 51

Before moving to Combinatorics / Counting, make sure you can:

- Compute GCD using Euclidean algorithm
- Compute LCM safely
- Explain GCD of strings

- Check if number is prime
 - Use Sieve of Eratosthenes
 - Do prime factorization
 - Use modulo addition/subtraction/multiplication
 - Use fast exponentiation
 - Understand modular inverse conceptually
 - Solve Pow(x, n)
 - Solve Count Primes
 - Solve Trailing Zeroes
 - Solve Sqrt(x)
 - Recognize GCD-based proof problems
-

44. Final Cheat Sheet

GCD

```
while (b != 0) {  
    int r = a % b;  
    a = b;  
    b = r;  
}
```

LCM

```
lcm = a / gcd(a, b) * b;
```

Prime Check

```
for (long long d = 2; d * d <= n; d++)
```

Sieve

```
for (long long p = 2; p * p <= n; p++) {  
    if (isPrime[p]) {  
        for (long long m = p * p; m <= n; m += p) {  
            isPrime[m] = false;  
        }  
    }  
}
```

Modular Addition

```
(a + b) % MOD
```

Modular Subtraction

```
(a - b + MOD) % MOD
```

Modular Multiplication

```
(1LL * a * b) % MOD
```

Fast Power

```
while (n > 0) {
    if (n & 1) {
        result *= base;
    }

    base *= base;
    n >>= 1;
}
```

Modular Power

```
result = result * base % MOD;
base = base * base % MOD;
```

Trailing Zeroes

```
while (n > 0) {
    n /= 5;
    count += n;
}
```

Digital Root

```
num == 0 ? 0 : 1 + (num - 1) % 9
```

45. Core Takeaway

Math problems are about finding the hidden rule.

The most important tools are:

- GCD for divisibility
- LCM for alignment
- sieve for many primes
- fast power for large exponents
- modulo arithmetic for huge answers
- factor counting for divisor problems
- binary search for integer square root
- formulas for digit and factorial problems

In interviews, always explain the math proof briefly.

A short proof often matters more than the code.
